



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programacion Dinamica



24 de mayo de 2026

Matias Urbani
113739

Paula Romano
112609

Juan Contino
113861

1. Introducción

Luego de haber analizado a todos los rivales gracias a tu ayuda, Scaloni definió un cronograma de entrenamiento. Tiene definido qué hacer para cada día de acá al mundial que viene, e incluso más. Para hacerlo más simple, para los próximos n días. El entrenamiento del día i demanda una cantidad de esfuerzo e_i , y podemos decir que la ganancia que nos da dicho entrenamiento es igual al esfuerzo. El entrenamiento que corresponde al día i (así como su esfuerzo y ganancia) son inamovibles: el Chiqui Tapia alquiló las herramientas específicas para cada día, y la AFA está muy ocupada organizando el torneo de 2^{30} equipos del año que viene para andar moviendo cosas. Si la cantidad de energía que se tiene para un día i es $j < e_i$, entonces la ganancia que se obtiene en ese caso es justamente j . (si se tiene más energía que e_i , no es que se pueda usar más para tener más ganancia).

A su vez, los jugadores no son máquinas. La cantidad de energía que tienen disponible para cada día va disminuyendo a medida que pasan los entrenamientos. Suponiendo que los entrenamientos empiezan con los jugadores descansados, el primer día luego de dicho descanso los jugadores tienen energía s_1 . El segundo día luego del descanso tienen energía s_2 , etc... Para cada día hay una cantidad de energía, y podemos decir que $s_1 s_2 \dots s_n$. Scaloni puede decidir dejarlos descansar un día, haciendo que la energía "se renueve" (es decir, el próximo entrenamiento lo harían con energía s_1 nuevamente, siguiendo con s_2 , etc...). Obviamente, si descansan, el entrenamiento de ese día no se hace (y no se consigue ninguna ganancia).

Scaloni no sabe bien cómo hacer para definir los días que deba entrenarse y los días que convenga descansar de tal forma de tener la mayor ganancia posible (y tener mayores probabilidades de ganar el mundial que viene), pero Menotti, exponente del juego bonito en Argentina, le recomendó usar Programación Dinámica para resolver este problema. Nos está pidiendo ayuda para poder resolver este problema.

2. Analisis del problema:

En el problema planteado, se nos pide determinar que días conviene entrenar y que días conviene descansar, y resolverlo mediante programación dinámica.

Sean E_i el entrenamiento que se puede realizar en el i -ésimo día, y S_j , la energía que se tiene después de j días sin descansar

Sea $D_{i,j}$ el día i , con la energía S_j

La energía que se puede ganar en $D_{i,j}$ es

$$G_{i,j} = \min(E_i, S_j) \quad (1)$$

Pero para poder hallar el optimo de $D_{(i,j)}$, llamado $Opt_{(i,j)}$, no basta ver cuanto podemos ganar hoy, ademas debemos ver que pasa con los días posteriores a i , y cual convendría mas entre descansar hoy (y tener mañana la energía de S_1) o entrenar hoy (ganando $G_{(i,j)}$ pero teniendo mañana menos energía)

Proponemos la siguiente ecuación de recurrencia

Caso base: ultimo día (día igual a n)

$$Opt_{(n,j)} = G_{(n,j)} \quad (2)$$

Caso General: resto de casos

$$Opt_{(i,j)} = \max(Opt_{(i+1,0)}, Opt_{(i+1,j+1)} + G_{(i,j)}) \quad (3)$$

y donde $Opt_{(1,0)}$ representa la ganancia máxima que se puede obtener con esa combinación de días

Proponemos un algoritmo de programación dinámica que revise todas las combinaciones de $Opt_{i,j}$ con un enfoque bottom-up, recordando todas las combinaciones ya revisadas para no perder información.

3. Demostración de optimalidad de la ecuación de recurrencia

3.1. Definición del problema y variables

Sea n la cantidad total de días de entrenamiento planificados. Para cada día $e \in \{0, 1, \dots, n-1\}$ (desde 0), se tiene un esfuerzo requerido $E[e]$, que representa tanto el esfuerzo necesario como la ganancia máxima posible de ese entrenamiento. A su vez, $S[z]$ denota la energía disponible cuando se llevan z días consecutivos entrenados desde el último descanso, con $z \in \{0, 1, \dots, n\}$, cumpliéndose $S[0] \geq S[1] \geq \dots \geq S[n-1]$.

La ganancia efectiva al entrenar el día e habiendo entrenado ya z días consecutivos es $\min(E[e], S[z])$: si la energía disponible no alcanza el esfuerzo requerido, la ganancia es simplemente la energía con la que se cuenta; si la supera, se obtiene la ganancia completa del entrenamiento.

Definimos $Pd(e, z)$ como la ganancia máxima obtenible desde el día e en adelante, que llevan z días consecutivos entrenados desde el último descanso. Esta función satisface la siguiente ecuación de recurrencia, que resume el razonamiento y el algoritmo, y cuya optimalidad se demuestra en la sección siguiente.

Ecuación de recurrencia

$$Pd(e, z) = \begin{cases} 0 & \text{si } e = n \\ \max \begin{cases} Pd(e+1, 0) & \text{(descansar)} \\ \min(E[e], S[z]) + Pd(e+1, z+1) & \text{(entrenar)} \end{cases} & \text{si } e < n \end{cases}$$

La primera opción del $\max$ refleja la decisión de descansar: no se obtiene ganancia en el día e y el contador de días consecutivos se reinicia a 0 para el día siguiente. La segunda opción refleja la decisión de entrenar: se acumula la ganancia $\min(E[e], S[z])$ y el contador pasa a $z+1$. La solución al problema completo es $Pd(0, 0)$, partiendo desde el primer día con los jugadores descansados.

3.2. Demostración

La demostración procede analizando los días restantes de atrás hacia adelante: primero se verifica el caso más sencillo (cuando no quedan días), y luego se argumenta que si la función es correcta para todos los estados futuros, también lo es para el estado actual.

Cuando no quedan días: $e = n$

Si $e = n$, no hay ningún día más por planificar. La única ganancia posible es 0, sin importar cuántos días consecutivos se lleven entrenados. La ecuación define $Pd(n, z) = 0$ para todo z , lo cual es correcto.

Supuesto sobre los días restantes

Supongamos que para todo día $e' > e$ y todo $z' \in \{0, \dots, n\}$, $Pd(e', z')$ ya contiene la ganancia máxima real desde el día e' con z' días consecutivos. Bajo este supuesto, hay que ver que $Pd(e, z)$ también es correcto.

Análisis de cada día

Sea G^* la ganancia máxima real que puede obtenerse desde el día e con z días consecutivos entrenados, es decir, la mejor ganancia posible considerando todos los planes de entrenamiento

válidos a partir de ese estado. En el día e , Scaloni tiene exactamente dos opciones:

Opción 1: descansar el día e . Si se descansa, no se obtiene ganancia en ese día y el estado del día siguiente es $(e + 1, 0)$. La ganancia total que puede lograrse a partir de aquí es la que se obtiene desde el día $e + 1$ partiendo descansado, que por el supuesto anterior es exactamente $Pd(e + 1, 0)$.

Opción 2: entrenar el día e . Si se entrena, se obtiene ganancia $\min(E[e], S[z])$ en ese día y el estado del día siguiente es $(e + 1, z + 1)$. La ganancia total que puede lograrse a partir de aquí es $\min(E[e], S[z]) + Pd(e + 1, z + 1)$, nuevamente por el supuesto anterior.

Combinando ambas opciones. Dado que estas son las únicas dos decisiones posibles, G^* es necesariamente el mayor valor entre ambas:

$$G^* = \max\left(Pd(e + 1, 0), \min(E[e], S[z]) + Pd(e + 1, z + 1)\right) = Pd(e, z)$$

Esto muestra que $Pd(e, z)$ coincide con la ganancia máxima real para el estado (e, z) .

Todo esto funciona porque el par (e, z) captura toda la información necesaria para tomar la decisión en el día e : el índice e determina el entrenamiento del día y su esfuerzo $E[e]$, mientras que z determina la energía disponible $S[z]$. No hace falta recordar el historial completo de decisiones anteriores; alcanza con saber en qué día se está y cuántos días consecutivos se llevan entrenados.

En conclusión, aplicando este razonamiento para todos los estados desde $e = n$ hacia atrás hasta $e = 0$, se obtiene que $Pd(e, z)$ es la ganancia máxima posible para cualquier estado válido (e, z) .

Ejemplo de Resolución

4. Algoritmo:

4.1. Código:

A continuación se muestra el código de solución por Programación Dinámica del problema.

```
def crear_matriz(dias, energias):
    n = len(dias)
    matriz = [[0] * (n+1) for _ in range(n+1)]

    for fila in range(n+1):
        matriz[fila][n] = 0 # caso base: no quedan dias

    for col in range(n-1, -1, -1):
        for fila in range(col+1):
            matriz[fila][col] = max(
                matriz[0][col+1], # descansar
                ganancia_hoy(dias, col, energias, fila) + matriz[fila+1][col+1] # entrenar
            )
    return matriz

def ganancia_hoy(dias, i, energias, j):
    return min(dias[i], energias[j])

def reconstruccion(energias, dias, Matriz):
    n = len(dias)
    res = []
    z = 0 # dias consecutivos entrenando
    e = 0 # dia actual

    while e < n:
        descanso = Matriz[0][e+1]
        entrenar = ganancia_hoy(dias, e, energias, z) + Matriz[z+1][e+1]

        if entrenar >= descanso:
            res.append("Entreno")
            z += 1
        else:
            res.append("Descansar")
            z = 0
        e += 1

    return res
```

4.2. Complejidad del algoritmo principal:

La matriz tiene dimensiones $(n + 1) * (n + 1)$. Por cada celda, se realizan $O(1)$ operaciones. El total de celdas es $(n + 1)^2$, por lo tanto: la complejidad temporal: $O(n^2)$, y la complejidad espacial: $O(n^2)$ para almacenar la matriz

4.3. Complejidad del algoritmo de reconstrucción:

Recorre los n días exactamente una vez. En cada día se realiza $O(1)$ operaciones. La complejidad temporal: $O(n)$ y la complejidad espacial: $O(n)$ para almacenar el plan resultante

5. Anexo: Analisis del problema:

En el problema planteado, se nos pide determinar que días conviene entrenar y que días conviene descansar, y resolverlo mediante programación dinámica.

Sean E_i el entrenamiento que se puede realizar en el i -ésimo día, y S_j , la energía que se tiene después de j días sin descansar

Sea $D_{i,j}$ el día i , con la energía S_j

La energía que se puede ganar en $D_{i,j}$ es

$$G_{i,j} = \min(E_i, S_j) \quad (4)$$

Pero para poder hallar el optimo de $D_{i,j}$, llamado $Opt_{(i,j)}$, no basta ver cuanto podemos ganar hoy, ademas debemos ver que pasa con los días posteriores a i , y cual convendría mas entre descansar hoy (y tener mañana la energía de S_1) o entrenar hoy (ganando $G_{(i,j)}$ pero teniendo mañana menos energía)

Proponemos un algoritmo de programación dinámica que revise todas las combinaciones de $Opt_{i,j}$ con un enfoque bottom-up, recordando todas las combinaciones ya revisadas para no perder información.

6. Anexo: Definición del problema y variables

Sea n la cantidad total de días de entrenamiento planificados. Para cada día $e \in \{0, 1, \dots, n-1\}$ (desde 0), se tiene un esfuerzo requerido E_e , que representa tanto el esfuerzo necesario como la ganancia máxima posible de ese entrenamiento. A su vez, S_z denota la energía disponible cuando se llevan z días consecutivos entrenados desde el último descanso, con $z \in \{0, 1, \dots, n\}$, cumpliéndose $S_0 \geq S_1 \geq \dots \geq S_{(n-1)}$.

La ganancia efectiva al entrenar el día e habiendo entrenado ya z días consecutivos es $\min(S_z, E_e)$: si la energía disponible no alcanza el esfuerzo requerido, la ganancia es simplemente la energía con la que se cuenta; si la supera, se obtiene la ganancia completa del entrenamiento.

Definimos $Pd(e, z)$ como la ganancia máxima obtenible desde el día e en adelante, que llevan z días consecutivos entrenados desde el último descanso. Esta función satisface la siguiente ecuación de recurrencia, que resume el razonamiento y el algoritmo, y cuya optimalidad se demuestra en la sección siguiente.

Anexo: Ecuación de recurrencia

$$Pd(z, e) = \begin{cases} 0 & \text{si } e = n \\ \max \begin{cases} Pd(0, e+1) & \text{(descansar)} \\ \min(S_z, E_e) + Pd(z+1, e+1) & \text{(entrenar)} \end{cases} & \text{si } e < n \end{cases}$$

La primera opción del $\max$ refleja la decisión de descansar: no se obtiene ganancia en el día e y el contador de días consecutivos se reinicia a 0 para el día siguiente. La segunda opción refleja la decisión de entrenar: se acumula la ganancia $\min(E_e, S_z)$ y la energía pasa a $S(z+1)$. La solución al problema completo es $Pd(0, 0)$, partiendo desde el primer día con los jugadores descansados.

7. Anexo: Demostración

La demostración procede analizando los días restantes de atrás hacia adelante: primero se verifica el caso más sencillo (cuando no quedan días), y luego se argumenta que si la función es correcta para todos los estados futuros, también lo es para el estado actual.

Cuando no quedan días: $e = n$

Si $e = n$, no hay ningún día más por planificar. La única ganancia posible es 0, sin importar cuántos días consecutivos se lleven entrenados. La ecuación define $Pd(z, n) = 0$ para todo z , lo cual es correcto.

Supuesto sobre los días restantes

Supongamos que para todo día $e' > e$ y todo $z' \in \{0, \dots, n\}$, $Pd(z', e')$ ya contiene la ganancia máxima real desde el día e' con z' días consecutivos. Bajo este supuesto, hay que ver que $Pd(e, z)$ también es correcto.

Análisis de cada día

Sea G^* la ganancia máxima real que puede obtenerse desde el día e con z días consecutivos entrenados, es decir, la mejor ganancia posible considerando todos los planes de entrenamiento válidos a partir de ese estado. En el día e , Scaloni tiene exactamente dos opciones:

Opción 1: descansar el día e . Si se descansa, no se obtiene ganancia en ese día y el estado del día siguiente es $(e + 1, 0)$. La ganancia total que puede lograrse a partir de aquí es la que se obtiene desde el día $e + 1$ partiendo descansado, que por el supuesto anterior es exactamente $Pd(0, e + 1)$.

Opción 2: entrenar el día e . Si se entrena, se obtiene ganancia $\min(E_e, S_z)$ en ese día y el estado del día siguiente es $(z + 1, e + 1)$. La ganancia total que puede lograrse a partir de aquí es $\min(E_e, S_z) + Pd(z + 1, e + 1)$, nuevamente por el supuesto anterior.

Combinando ambas opciones. Dado que estas son las únicas dos decisiones posibles, G^* es necesariamente el mayor valor entre ambas:

$$G^* = \max\left(Pd(0, e + 1), \min(E_e, S_z) + Pd(z + 1, e + 1)\right) = Pd(z, e)$$

Esto muestra que $Pd(z, e)$ coincide con la ganancia máxima real para el estado (e, z) .

Todo esto funciona porque el par (z, e) captura toda la información necesaria para tomar la decisión en el día e : el índice e determina el entrenamiento del día y su esfuerzo E_e , mientras que z determina la energía disponible después de z días sin entrenar S_z . No hace falta recordar el historial completo de decisiones anteriores; alcanza con saber en qué día se está y cuántos días consecutivos se llevan entrenados.

En conclusión, aplicando este razonamiento para todos los estados desde $e = n$ hacia atrás hasta $e = 0$, se obtiene que $Pd(e, z)$ es la ganancia máxima posible para cualquier estado válido (e, z) .

8. Anexo: Algoritmo:

A continuación se muestra el código de solución por Programación Dinámica del problema.

```
def crear_matriz(dias, energias):
    n = len(dias)
    matriz = [[0] * (n+1) for _ in range(n+1)]

    # Llenado de derecha a izquierda
    for día_actual in range(n-1, -1, -1):
        for días_consecutivos in range(día_actual+1):
            matriz[días_consecutivos][día_actual] = max(
                matriz[0][día_actual+1], # descansar
                ganancia_hoy(dias, día_actual, energias, días_consecutivos) + matriz[días_consecutivos+1][día_actual+1]
            )

    return matriz

def ganancia_hoy(dias, i, energias, j):
    return min(dias[i], energias[j])

def reconstruccion(energias, dias, Matriz):
    n = len(dias)
    res = []
    z = 0 # días consecutivos entrenando
    e = 0 # día actual

    while e < n:
        descanso = Matriz[0][e+1]
        entrenar = ganancia_hoy(dias, e, energias, z) + Matriz[z+1][e+1]

        if entrenar >= descanso:
            res.append("Entreno")
            z += 1
        else:
            res.append("Descansar")
            z = 0

        e += 1

    return res
```

9. Anexo: Variabilidad de los valores:

La variabilidad de los no afecta la complejidad, siempre es $O(n^2)$ para el PD y la reconstrucción recorre n días sin importar los valores por lo tanto es $O(n)$, la cantidad de celdas a recorrer es independientemente de los valores. Sin embargo, los valores si pueden modificar la solución obtenida. Por ejemplo, si la energía se gasta mas rápido probablemente convenga descansar antes mientras que si el esfuerzo es mas bajo, probablemente se entrenaran mas días seguidos. esto afecta la solución pero no la Optimalidad.

10. Anexo: Complejidad

10.1. Complejidad del algoritmo principal:

La matriz tiene dimensiones $(n + 1) * (n + 1)$. Por cada celda, se realizan operaciones $O(1)$. El total de celdas es $(n + 1)^2$, por lo tanto: la complejidad temporal: $O(n^2)$, y la complejidad espacial: $O(n^2)$ para almacenar la matriz

10.2. Complejidad del algoritmo de reconstrucción:

Recorre los n días exactamente una vez. En cada día se realiza operaciones $O(1)$. La complejidad temporal: $O(n)$ y la complejidad espacial: $O(n)$ para almacenar el plan resultante

11. Anexo: Ejemplos de ejecución para encontrar soluciones

Se analizaron varios casos para corroborar que el algoritmo es optimo en determinadas circunstancias:

11.1. Caso base:

Entrenamientos	Energías
10	7
12	5
9	4

El entrenamiento total es 16

Es un caso normal para evaluar que el algoritmo funciona correctamente

11.2. caso variados:

Entrenamientos	Energías
10	99
70	20
80	5
15	1

El entrenamiento total es 105

Un caso muy variado, con tiempos de entrenamiento y energías grandes y pequeñas

11.3. caso Energías Iguales:

Entrenamientos	Energías
19	5
13	5
15	5
12	5
10	5

El entrenamiento total es 25

Con energías iguales, el entrenamiento total pasa a depender principalmente de la capacidad de entrenamiento de cada día, pues no supone ninguna ganancia descansar

11.4. caso Entrenamientos iguales:

Entrenamientos	Energías
10	12
10	11
10	10
10	8
10	6
10	4
10	3
10	2

El tiempo total es 68

Con capacidad de entrenamientos iguales, el entrenamiento total pasa a depender de las energías

12. Anexo: Conclusión

A lo largo de este trabajo se abordó el problema de planificación de entrenamiento de la selección argentina mediante la técnica de Programación dinámica. Se propuso una ecuación de recurrencia, se demostró su optimalidad y se implementó un algoritmo que la resuelve eficientemente. El algoritmo de cálculo de óptimos tiene complejidad temporal $O(n^2)$ y espacial $O(n^2)$ derivadas directamente de la matriz de dimensiones $(n + 1) \times (n + 1)$ que se construye en el enfoque bottom-up. El algoritmo de reconstrucción tiene una complejidad $O(n)$ en tiempo y espacio, ya que recorre los n días exactamente una vez. Estas complejidades son independientes de los valores de E y S , el número de operaciones está determinado únicamente por n . La Programación Dinámica permite explorar implícitamente el espacio de 2^n estrategias posibles en tiempo polinomial, garantizando la optimalidad de la solución